Date of publication xxxx 00, 0000, date of current version November 16, 2025.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

UAV Drone Collision Detection and Prevention using LiDAR

JACOB BRENDEMUEHL[1], OLIVIA GETTE[2], and FELICIA HUFFMAN [3], and SAMUEL WEISS[4]
[1]College of Computing, Michigan Technological University, Houghton, MI 49931 USA (e-mail: jkbrende@mtu.edu)
[2]College of Computing, Michigan Technological University, Houghton, MI 49931 USA (e-mail: omgette@mtu.edu)
[3]College of Computing, Michigan Technological University, Houghton, MI 49931 USA (e-mail: fmhuffma@mtu.edu)
[4]College of Computing, Michigan Technological University, Houghton, MI 49931 USA (e-mail: saweiss@mtu.edu)

This work was sponsored by Michigan Technological University

ABSTRACT This project was designed and implemented with the objective of using reinforcement learning to train a simulated MK 30 Amazon Delivery Drone to use only LiDAR for its navigation purposes. To do this, we simulated a real-world environment in Unity with expected hazards such as weather conditions and obstacles (static and dynamic). We then built a simulated drone with realistic sensors and actuators. Finally, we used reinforcement learning to detect and avoid colliding while navigating the environment and 'delivering' a package to a given 'goal', which was the porch of a given house in our environment. Our project helped broaden the research knowledge of LiDAR-based inputs to reinforcement learning algorithms. Previous studies have involved set-algorithms with specified inputs and outputs. We found that it is possible for a UAV drone to be taught to navigate an unknown environment with only LiDAR-based sensors and a reinforcement learning pipeline.

INDEX TERMS UAV, Drone, LiDAR, Reinforcement Learning

I. INTRODUCTION AND BACKGROUND

UNMANNED aerial vehicles (UAVs) have become a hot topic recently, with Amazon planning to use UAV drones to deliver packages and many militaries planning to, or currently using them for warfare, as well as many other applications. While UAVs have increased efficiency, cost, safety, and flexibility, there are still many challenges being faced due to the safety hazards involved in navigation and control. This stems in part from their minimal sensor arrays, since saving on cost is one of the large appeals of an individual drone. In addition to these challenges, companies are trying to overcome the negative public perception of UAVs since many current UAVs have cameras on them for navigation purposes. In this project, we aimed to gain a better understanding of Light Detection and Ranging (LiDAR) and how it could potentially help narrow the gap between today's UAVs and the future of UAVs in the area of package delivery. [1]

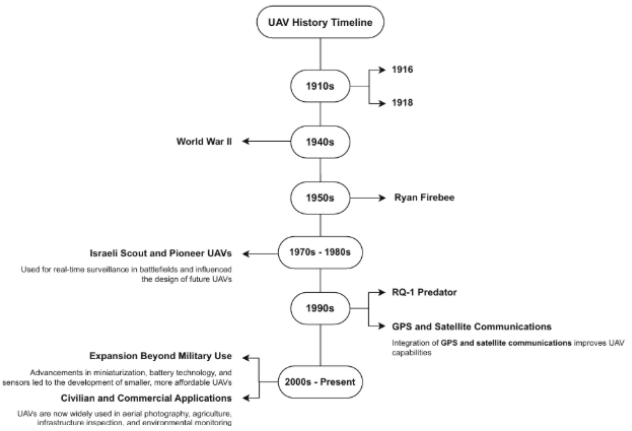

FIGURE 1. Timeline of UAvs, early 1900s - Present

A. HISTORY OF UAVS

UAVs first recorded use was in 1916, when the US Navy constructed the Aerial Target, a drone meant to carry and drop explosives on a designated target. Later in 1918, the US Army developed the Kettering Bug, a flying bomb. These two UAVs laid the groundwork for UAV use in World War II, such as the German V-1 flying bomb and the US TDR-1 assault drone. The Cold War in the mid to late 20th century led to even more development and testing of UAVs, especially with the advancements in electronics and control systems in the 70s and 80s. In the 20th century, UAVs were mainly used for military purposes and were not commonly seen in other applications. [2]

When the 21st century hit, many technological improvements led to increased use cases for UAVs such as aerial photography, agriculture, infrastructure inspection, and environmental monitoring. You can see the timeline of this in Figure 1, which was sourced from "Drone and UAV Forensics" by Hudan Studiawan and Kim-Kwang Raymond Choo. Today, UAVs are being considered for use in many artificial intelligence applications, such as our drone delivery example. Large companies are hoping to utilize this technology as it has recently become more accessible, practical, and feasible. [2]

It is important to note that UAVs are not drones and drones are not UAVs. However, in this paper we often reference UAVs when we are discussing our drone. Assume all references to a UAV is referring to a drone where there are no controls other than sensor input and action output.

B. HISTORY OF LIDAR

LiDAR used as an object detection method involves sending several laser beams out and measuring the reflection of them to determine the distance and location of objects around you. This can be determined through the variation in wavelength and how long it takes to fully reflect back on your LiDAR sensor. This technology has been around for decades, but is only recently being combined with artificial intelligence and machine learning approaches for navigation purposes. LiDAR in the form of light pulses was first used in the 1930s to determine the height of clouds. Around 1961, an early version of LiDAR, known as Colidar (Coherent light detecting and ranging) was developed for tracking airplanes shortly after the invention of the laser. From here, Colidar made its way into military applications, was used by NASA to collect data on the ocean water and atmosphere, and enhanced topography mapping capabilities. Lidar was fully developed later in the 1980s, when the push for global positioning systems led to a need for a sensor in aerial photogrammetry, which LiDAR proved to be best at. [3]

LiDAR applications have grown rapidly since the 80s, with use cases in agriculture, archaeology, autonomous vehicles, atmosphere, law enforcement, military, mining, robotics, surveying, forestry, and now in UAVs. This newer application is most similar to autonomous vehicles; however, they pose new challenges, such as the potential for obstacles in every direction, meaning there is potential to collide left, right, forward, and backward with an autonomous vehicle, but a UAV also includes the challenge of moving up and down. This makes alternate sensor options like cameras or radar more costly and inefficient since it would require a spherical view, which is why LiDAR research is so important for UAVs in general. Recent papers have concluded that LiDAR sensors are the future, but with some of the safety and navigation concerns in their application to AI/ML fields, more research is needed before they can be fully implemented in practice. [3]

C. PROJECT PURPOSE

UAVs and LiDARs are two technologies that have many applications across a variety of fields, but more research is needed to address specific challenges in the ethical navigation of these drones in the real world. We need a way to ensure that the UAV can, without human guidance, navigate a real-world environment safely without being a hazard to people, homes, vehicles, and animals. UAV adoption in a variety of systems, but especially in package delivery, could be beneficial to the environment and could lead to less traffic on the road. To address the need for safe systems and the lack of LiDAR research with respect to UAV drones, we constructed this project to help improve and raise awareness regarding this research gap.

II. RELATED WORK

There have been several studies conducted that discuss the feasibility of using LiDAR in UAV drones to detect and avoid collision with various obstacles including birds and malicious drones. An article by (Paula Seoane et. al) performs a study on drones ability to sense and react to birds, which found that they can be detected at about 30m. However, they saw increased error when the drones speed differed more greatly from the speed of the birds. They concluded that the LiDAR detection system could be promising for detection of objects in the air, especially as the objects get larger and easier to detect with LiDAR. [4]

Another article explored the potential for collision detection and avoidance with malicious drones using LiDAR-based sensors. This paper mentions the importance of developing drones with a higher level of autonomy since some UAVs are still overseen to ensure safe operations. In attempt to further research on the future autonomous drones, a study was conducted to see if an obstacle detection and avoidance algorithm could be implemented given the LiDAR input. They found that the precision of object detection decreased as the distance increased and less point clouds of LiDAR were produced. The collision avoidance algorithm was successful in determining the optimal maneuver to successfully avoid the collision. This showed that it was possible for a UAV to avoid collision with other flying objects using LiDAR sensors. [5]

An article focusing more on the safe travel of a UAV in regards to a flight path with a specified goal uses LiDAR to obtain more information about its environment before providing it to algorithms to determine the desired speed of the drone and how it can avoid the obstacles detected. To test this, they used a simulated forest environment and ran flight path simulations in it where their UAV collected the lidar data, sent it to the obstacle avoidance algorithm, then reacted according to it. They mention that while the UAV was successful in avoiding obstacle collisions, it was not very efficient and lacked the ability to find the optimal path to its goal. The researchers claimed adding an optimal global search algorithm such as A* could improve this issue. [6]

Our work with drone collision avoidance differs from the prior three articles, and many articles written by being

focused on a reinforcement learning approach to the problem rather than an algorithmic approach. The benefits of reinforcement learning over an algorithm is that it has the potential to generalize better and adapt more quickly to its environment and changing scenarios. It also has the potential to perform better on edge cases that the developers may not have thought of. Unfortunately, it is difficult for us to show this since we are not training the drone in a real-world environment, but theoretically it makes sense. We also actively address the issue of inefficient paths by penalizing living time and directions facing away from the goal. Reinforcement learning, unlike algorithms, can focus on the bigger picture: obstacle avoidance, efficient paths, and safe flight paths simultaneously.

III. METHODOLOGY

A. PROJECT IMPLEMENTATION PLAN

Since our project was completed as a group of four, we decided early on that we would need to implement agile management processes in order to be successful in our project. We used the digital Kanban board service Jira, as well as GitHub for version control/file sharing. We held bi-weekly meetings where we would review tasks and add items to the Jira where we could make deadlines, set priorities, and keep track of our progress. Additional meetings would be scheduled when we were trying to meet deadlines or needed assistance with a difficult portion of the project. [7][8]

We mostly used the software Unity to simulate and manipulate the environment with the exception of the drone which was modeled in Blender for greater accuracy. We also utilized a shared Google Drive along with our Github and Jira board to keep track of any files that were being worked on simultaneously. Anaconda was used to ensure library/package version compatibility across our team member's computers. Jupyter Hub was used to manipulate and visualize our data using Python. [9][10][11][12][13]

In future projects with Unity, we caution other teams to use Unity's built-in version control system rather than Github to avoid some of the challenges we faced. There were issues with our GitHub repo throughout the project because Unity creates large files which must be included in a .gitignore as well as temporary files that cause merge conflicts very frequently. Because of this and other concerns of losing progress, we decided to create two local copies of the project as backups. Between the four of us, this method proved to be quite useful at times and ensured we never lost significant progress. In addition, we did not find creating software branches to be of much use, further calling GitHub's usefulness into question.

B. SOFTWARES USED

Our reinforcement learning agent and environment setup were created using multiple available softwares. Unity (version 6000.2.2f1) is our physics simulator engine where we developed and constructed the majority of our project. The machine learning itself is done using the Unity library ML-Agents (version 1.1.0) which is based on Barracuda, which is known to be useful for reinforcement learning. Both Python (version 3.10+) and Conda were also required to successfully run our project. Additionally, we used Jupyter Notebooks to explore preliminary data to help gain an understanding of the robotics world. Later in the project, Tensorflow dashboards were used to create reward plots to keep track of our performance. Throughout the entire project, we used GitHub version control to keep track of changes to our Unity environment, as well as documentation and other resources required for our project. Anything that was unable to, or not necessary to put in the repository went into a shared Google Drive.

C. DATA COLLECTION AND PREPROCESSING

Our data collection and preprocessing was quite difficult due to the lack of available research on LiDAR-based drone navigation systems. After digging through countless articles dealing with imaging and radar, we finally found a study through the University of Turku using a combination of LiDAR and image inputs to navigate a drone through an unfamiliar environment. The main purpose of this study was to collect a dataset that could be used for further research and advances on UAV tracking. [14]

The data was available through this study and was stored in Rosbag files which we downloaded and manipulated on a flashdrive. Using Anaconda, we ensured the files did not have any critical components and were a reasonable size before analyzing them further. In order for the files to be reasonably analyzed given our computing capabilities, we had to aggregate the data to make the files shorter, since each one was several gigabytes. To do this, we aggregated by taking every 10-20th row, depending on the file, then exported the files to Jupyter Notebook. After that, we were able to use interpolation to combine the LiDAR and position data for further analysis.

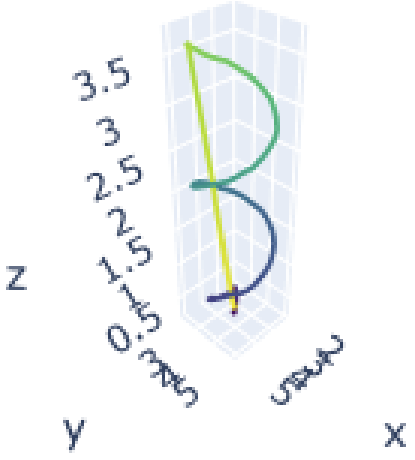

FIGURE 2. Trajectory of Spiral Data from Turkish Experiment

The main trajectory path we analyzed for insights was

the spiral data shown in Figure 3 above, since the drones position was moving in the x, y, and z directions. We found that the data included variables such as velocity, acceleration, position, and orientation of the drone as well as x, y, and z coordinates as well as reflectivity of the LiDAR sensors. This gave us insights into how we should construct our simulated drone and the importance of accurate physics values for drone movement and LiDAR based tracking. We decided that given the LiDAR inputs, the drone should be deciding not only what direction to go, but also the velocity and acceleration the drone should be moving. The main thing this preliminary analysis helped us discover was how to best simulate the drone as if it were in the real-world.

D. SYSTEM ARCHITECTURE

The UAV simulation environment is implemented entirely in Unity, and it serves as the core platform for testing drone behaviour, sensor performance, and control logic. The drone itself is modeled as a rigidbody-driven vehicle whose dynamics are controlled by Unity's physics engine. This allows a realistic simulation of lift, thrust, torque, gravity, and drag. The drone's structure includes a main body with colliders, six motor objects, and two LiDAR arrays that are mounted at the front and rear of the frame. The drone is shown in Figure 2. Each sensor uses a custom LiDARSensor script component that generates a hemispherical array of beams, which is displayed in Figure 3. This enables the UAV to perceive the surrounding 3D environment and report back to the drone. These rays can be spaced at angular intervals, such as 15, 30, or 45 degrees. This provides flexibility in controlling LiDAR resolution and performance cost. The environment itself is shown in Figure 4 and contains terrain, obstacles, walls, and other interactable surfaces that are defined under a layer used for raycasting. Laser hits and distances are recorded at timed intervals, which allows the drone to maintain an accurate sense of depth and local structure within the environment.

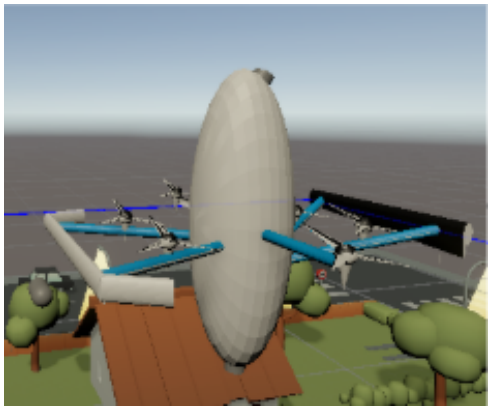

FIGURE 3. Drone simulated in Unity

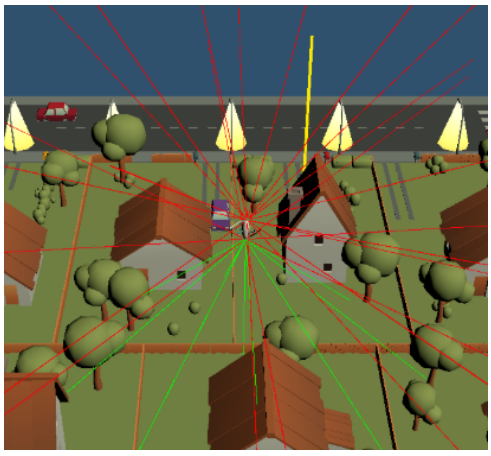

FIGURE 4. Lidar arrays from Unity simulation

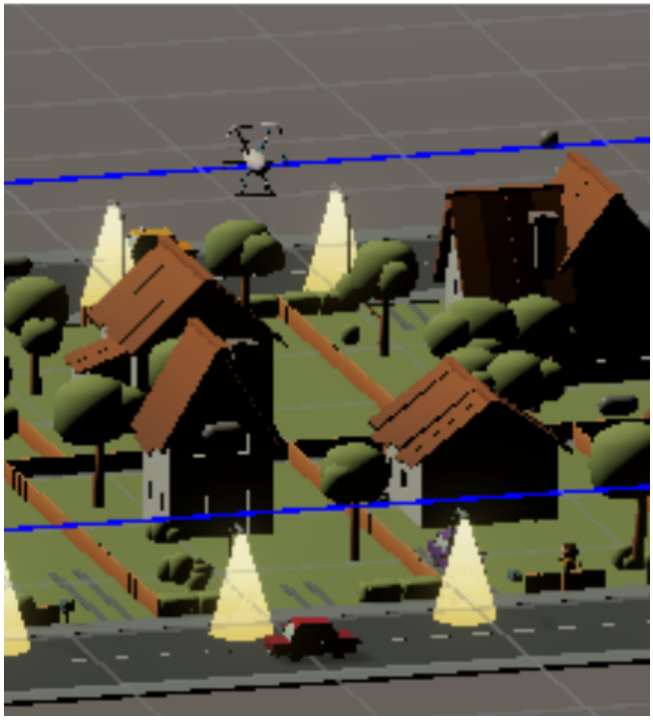

FIGURE 5. Unity environment with objects to avoid

To support detailed analysis, we created a logger script which captures the drone's state at each time step. For each beam, it displays the numbered beam, azimuth/elevation angles, and the Euclidean distance for the beam. It also records the strength of each motor based on the flight controller, which allows each sensor frame to be paired with exact control inputs that generated it. This data is written into a session-specific CSV file, organized into a dedicated logging folder which we called LiDAR Logs. Here, it can easily be retrieved and used for reinforcement learning purposes. The motor system itself is managed by a script we called MotorController that translates movement into physical forces applied to the drone's rigid body. Motor strengths determine lift and

torque, which in turn influence pitch (lateral rotation), roll (longitudinal axis rotation), yaw (vertical axis rotation), and altitude of the drone. Together, the physics engine, LiDAR sensors, motor controller, and logging subsystem create a cohesive virtual UAV platform capable of simulating flight dynamics, collision scenarios, and sensor-driven navigation tasks.

E. ALGORITHMIC APPROACH

The UAV is equipped with an array of sensors (including the LiDAR batteries) and is actuated by six propellers and two airfoils. If operated correctly, these components are sufficient to allow for safe and efficient flight. The algorithm which governs the drone's behavior is a simple feedforward pipeline of four elements. At each step, the simulation computes in a series of four scripts, titled Unity Environment, Drone Agent, Drone Autopilot, and Drone Root, which are described below:

1) Unity Environment

This script allows us to compute physics forces like thrust, gravity, and collisions when they occur. Most of the physics behavior we want to calculate is handled for us by the Unity environment, which is a convenience that saves us development time. We did, however, have to estimate the physics values related to our drone, such as mass, damping, and lift. We did this by referencing similar drones and any public information available through Amazon.

2) Drone Agent

The state information of the sensors is computed and fed to the DroneAgent as a vector. The values of this state information are timestamp in seconds, yaw, pitch, and roll in degrees, velocity in direction x, velocity in direction y, and velocity in direction z all in meters per second, acceleration in direction x, acceleration in direction y, and acceleration in direction z all in meters per second squared, the strength measurements of each of the six motors, and the distances in meters from each beam calculated in meters. Based on this observation vector, the drone's agent will make four decisions about how the UAV's motors should behave next. These decisions include pitch, roll, and yaw, which determine the rotation about each axis of the drone, and climb, which determines the necessary altitude increase of the drone. Each command is encoded as a value from -1 to 1, where larger magnitudes cause a more significant effect in their respective direction. This script also controls the learning/update behavior for the agent's neural network, which drives the reinforcement learning.

3) Drone Autopilot

The drone's "Autopilot" script has two main functions: To maintain balance, or keep the drone "upright": It uses the state information computed in the Drone Agent script to fight for balance using simple calculus equations. By increasing or decreasing thrust in the drone's motors, this script attempts to minimize the derivative value of the drone's pitch and roll, functionally maintaining balance. This first half of the script uses no machine learning whatsoever, and without further instruction would cause the drone to simply float in place indefinitely. The second half of this script exposes the four controls to the agent (pitch, roll, yaw, and climb) the values for which were computed earlier. For example, if the drone's agent were to output a -0.95 pitch, then the drone would tilt significantly clockwise about the X axis. But if the output was a pitch of 0.25, then the drone would tilt slightly counterclockwise about the X axis. The same logic applies the the other three controls.

4) Drone Root

Lastly, the script to tie in the above three, and control the drone itself. The commands computed in the earlier step are executed as adjustments to the drone's current motor thrust. These motors are articulated realistically, having a minimum and maximum Newton output value as well as having clamped spin up and spin down rates. After the drone's motor forces are computed, we then take the information learned from this pass and return to the Unity Environment script to recalculate our action given the new information.

F. REINFORCEMENT LEARNING (RL) APPROACH

For our reinforcement learning, we used the ML-Agents library designed for Unity. It is based on Python's Barracuda Library, a lightweight neural network inference library for Unity [14]. Among its requirements is a config file which specifies the neural network architecture, learning algorithm, and hyperparameters. Using this Application Programming Interface (API), we specified a relatively ordinary Multilayer perceptron (MLP) architecture for our drone's neural network. This choice was made to fit the small dimensional depth requirements of our agent's input and output vector size as well as fitting a constraint on our available compute power. If future teams were to replicate our work with access to higher compute, we would suggest using Long Short Term Memory (LSTM) architecture instead due to its superiority in time series tasks.

The remaining reinforcement learning tasks are handled by our agent script, which is tasked with the following:

- Dictating the beginning state of each episode including the location of all actors and obstacles
- Recording information about the environment during the episode, which is used for determining if a terminal condition is met, logging, and for later use by the reward function
- Ending each episode, checking for terminal conditions and cleaning up the environment for the next episode if one is reached
- Processing the input observation vector to the neural network which includes the timestamp, roll, pitch, and yaw, velocity and acceleration about the x, y, and z axis, motor strength for each of the six motors, and distance calculated from each beam.

- Map outputs which are based on the current input/observation vector described above. The resulting four dimensional output vector is fed to the script described previously called Drone Autopilot, effectively allowing the UAV to take actions in the simulator. Each index of the output vector corresponds to one of the four possible actions. These commands are followed by increasing or decreasing the thrust to a given rotor on the UAV as it has no other actuators
- Calculate the reward function for the agent to use in its policy update. This function rewards successful delivery of its package as well as swiftness, while punishing closeness to obstacles, meandering behavior, and collisions. The collision penalty is scaled by the linear impact velocity at the time of the collision, so severe crashes are penalized more heavily than light ones. This serves as the agent's sole feedback from its actions between episodes.

Note: The drones policy updates are abstracted from us, following a Deep Q Learning algorithm which was made available to us directly through the ML-Agents library.

IV. RESULTS AND DISCUSSION

A. QUANTITATIVE RESULTS

Training the drone allowed us to produce several measurable trends across reward, episode length, success indicators, and collisions. We visualized these using a TensorBoard dashboard that contains all this information in graph and numeric form.

The first metric we tracked is the total reward, shown in Figure 6. There is an Episode/Total Reward graph that shows an upward trend over time in the training. This is because earlier episodes produced highly negative rewards due to crashes and unstable flight, but as training progressed, the reward became more stable and trended towards higher values. This shows that the policy was able to slowly learn more controlled behaviors.

The second metric used is the episode length, shown in Figure 7. There is an Environment/Episode Length plot that shows a low episode length for the beginning episodes, which reflects more early crashes. As training continues though, we see that the episode lengths actually start to decrease. While this may seem concerning at first glance, this could actually be a positive sign. We have a couple of graphs that visualize the reasons why an episode ended, shown in Figure 8 and 9. Since we can see that no episodes actually ended due to a hard crash, we know that a majority of the episodes ended due to timeouts or the drone stopped improving. This adds extra, unnecessary time to the episodes. Over time, the number of episodes that ended due to timeouts or no improvement decreased significantly. This means that the most likely reason for the overall decrease in episode length is due to more frequent and quicker successes, showing the drone's continuous improvement over time.

We also had a metric that tracked whether or not the drone showed significant improvement across episodes. You can see

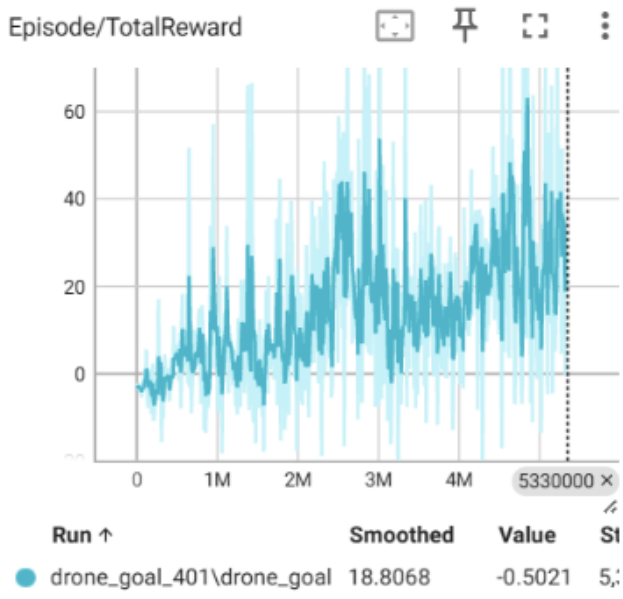

FIGURE 6. Total reward shown across episodes

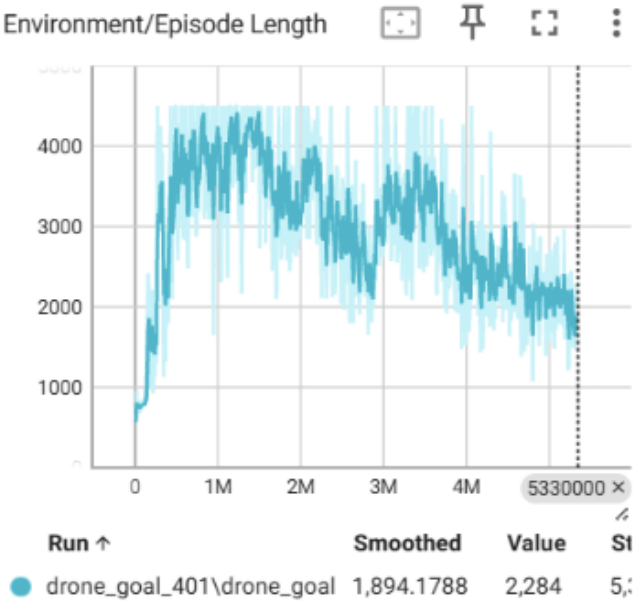

FIGURE 7. Learned Environment over episode length

in Figure 10 that the proportion of episodes that ended this way was significantly reduced over time, which is expected if the model is doing well.

The next metric we track is collisions. Figure 11 shows the number of episodes ended due to a hard crash. As mentioned earlier, there were actually no collisions severe enough to end an episode.

Lastly, we have a success metric that you can view in Figure 12. We had some challenges with the success metric correctly counting successful episodes, but were able to get it working

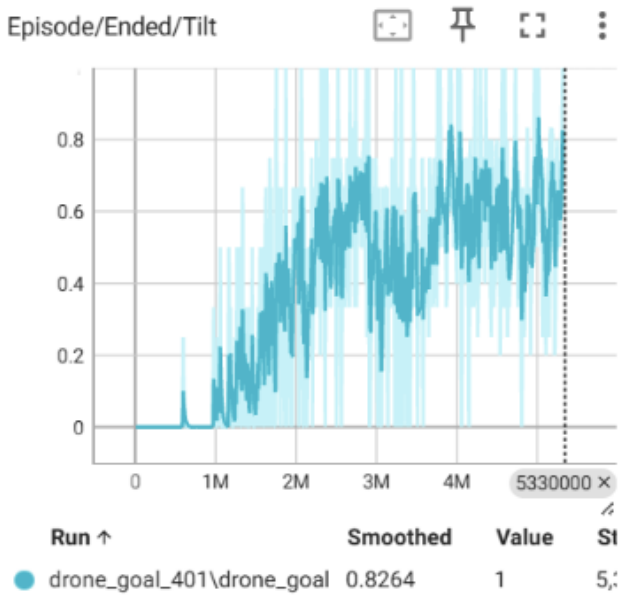

FIGURE 8. Episodes that ended due to too much tilt

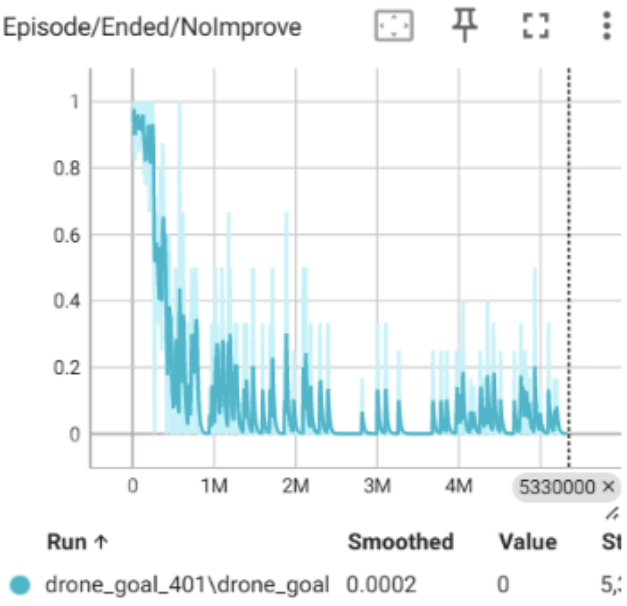

FIGURE 10. Episodes that ended in no improvement

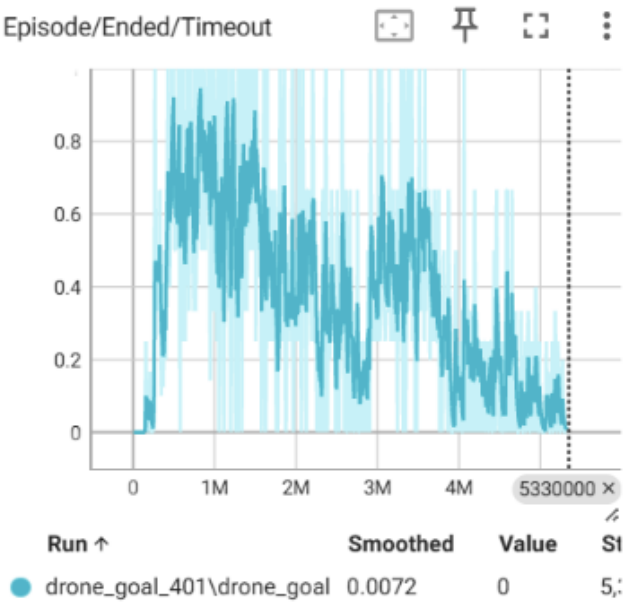

FIGURE 9. Episodes that ended due to timeout

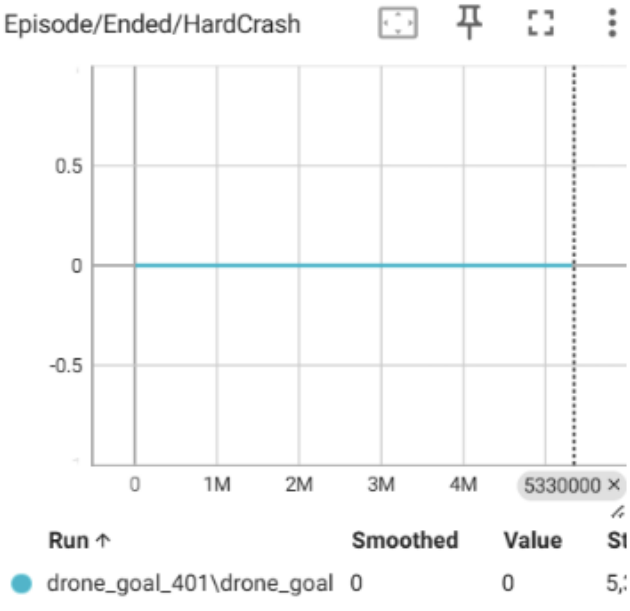

FIGURE 11. Episodes ending due to a hard crash

correctly. The graph shows a low number of successes starting off, but as time goes on, the number of episodes ending in success increases. This suggests that the drone is starting to learn how to safely find the goal.

B. QUALITATIVE RESULTS

The drone appears more physically stable and successful as time went on. At the beginning of a training session, the drone shows more oscillations and quicker crashes, which is to be expected in a reinforcement learning environment. As training continues though, although the oscillations didn't smooth completely, you can see the drone making better approaches towards the goal with less need for corrective action which is seen through the overall upward trend. We also noticed differences in the trajectories of the drone over time. As time goes on later into the training, the trajectories of the drones become less sporadic showing smoother, more intentional movements.

C. CHALLENGES

- High compute requirements for more complicated Neural Network architecture

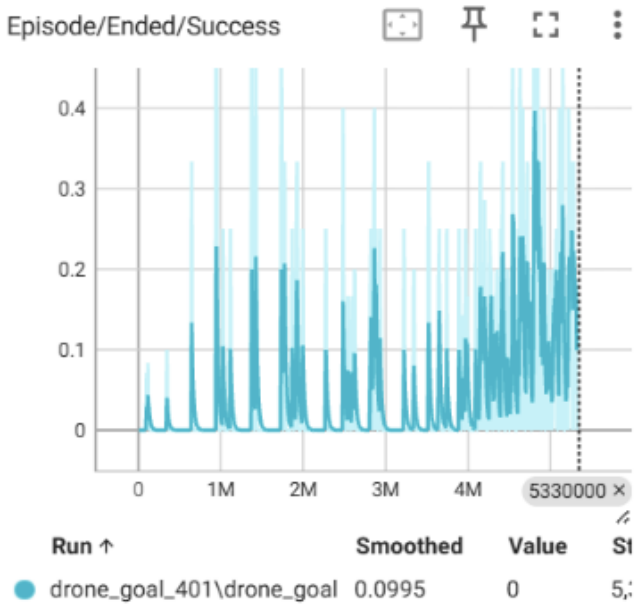

FIGURE 12. Episodes that ended in success

  - Unfortunately, we were unable to use the high computing machines available through Michigan Technological University because it did not support Unity builds. Because of this, we had to use our personal computers with significantly less computing power. This meant using a Multilayer Perceptron network, rather than the Long Short Term Memory network (LSTM) we would have preferred to use.
- Large waiting periods between changing the reward function and improved behavior
  - You have to run the training simulation in order to see the outcome of your changes, and in order to do that you need to run thousands of episodes. We unfortunately were not able to make any changes while it was running, so we had to make the change then run it to view the results. This added significant time to debugging.
- Designing an effective reward function without causing unintended behaviors
  - This is partly because there was such a long waiting period when changes were made. It was difficult because reinforcement learning is so particular, meaning small changes can make a big impact and it is difficult to know which variables need to be changed.
- Reward hacking/unexpected behavior
  - The drone would occasionally get close to the goal, then tilt towards it and earn a success to end the episode. This was the most clear example of reward hacking that occurred.
- Version control, software compatibility and other classic software development issues
  - In the future, we would use Unity's built in version control rather than GitHub. This would hopefully help resolve the issue of complex merge conflicts we encountered through GitHub.

D. POTENTIAL IMPROVEMENTS

Looking into the future, there are several ways we can strengthen and extend our project. One important step would be gaining access to high-powered computing resources. This would allow us to run more complex models and scale training to larger environments. This would not only accelerate convergence but also enable us to explore more advanced reinforcement learning algorithms. Also, adding a second drone to see how the model handles multiple drones in the same environment would be a potential improvement. This will add a more unpredictable obstacle that would be important to test if our drone were to come in contact with another drone at some point. Another possible improvement would be extending the algorithms to a real-world application by using it on a physical drone. Testing outside the simulation would validate whether the learning policy generalizes to unpredictable dynamics and environmental noise. Lastly, our end goal would be to expand the scope of the drone's mission by training it to not only deliver a package, but also to return to the hub, which would require higher level navigation and more efficient resource planning. Together, these improvements would make the training process more realistic, scalable, and aligned with practical deployment goals.

E. DISCUSSION OF OBSERVED PATTERNS

During training, the drone's behavior improved in several noticeable ways. The reward curve started increasing over time, which shows that the drone started to make fewer big mistakes and learned to make more consistent actions. Episode lengths also started to decrease as training continued, meaning the drone likely started to succeed quicker and therefore ending episodes quicker.

Watching the drone in Unity supported the results. Early in the training, the drone moved quite uncontrollably and often got stuck on obstacles trying to get to the goal. Later into training, it hovered more steadily, made smoother turns, and approached the target in a more controlled manner. The drone also showed more predictable flight paths, while timeouts happened less often.

Overall, the training patterns matched what we would expect from reinforcement learning. The drone has shown slow, but steady improvement as it interacts with the environment. Looking at and comparing all the different metrics, they suggest that the drone is learning meaningful behavior and not only increasing the reward. By continuing to improve upon what we have mentioned, we will likely be able to make the results even stronger.

V. CONCLUSION

In addition to the application in the area of drone delivery, this research can be expanded to apply to other LiDAR

based applications, such as military use. We have proved that through reinforcement learning, it is possible to train a drone to successfully conduct collision detection and avoidance using solely LiDAR based sensors.

In proving this, we created a Unity environment with multiple objects to act as potential hazards, simulated a drone using Blender and estimated physics values, and programmed the drone to use LiDAR input for navigation through reinforcement learning with rewards for making progress and punishment for backtracking or failure to finish the goal. Despite only training with a Multi-Layer Perceptron (MLP), we were able to achieve significant results that proved the drone was learning over time, with an overall improving success rate.

In the future, we would like to implement this algorithm and technique in a real-world drone to see how it reacts and learns in a real environment. Before that is possible, we would like to run the model on an Long Short Term Memory (LSTM) network for more reliable estimates and a longer run time.

REFERENCES

[1] I. Nurgaliev and Y. Eskander, "The use of drones and autonomous vehicles in logistics and delivery," *Logistics and Transport*, vol. 1–2(57–58), pp. 77–87, 2023. DOI: 10.26411/83-1734-2015-2-55-6-23.

[2] H. Studiawan and K.-K. R. Choo, "Introduction to drone and UAV technology," in *Drone and UAV Forensics*, Cham, Switzerland: Springer, 2025, pp. 1–19. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-93511-4_1

[3] N. Mehendale and S. Neoge, "Review on Lidar Technology," SSRN, May 18, 2020. [Online]. Available: https://ssrn.com/abstract=3604309

[4] P. Seoane, E. Aldao, F. Veiga-López, and H. González-Jorge, "Assessment of LiDAR-based sensing technologies in bird–drone collision scenarios," *Drones*, vol. 9, no. 1, p. 13, Jan. 2025. [Online]. Available: https://doi.org/10.3390/drones9010013

[5] E. Aldao, L. M. González-de Santos, and H. González-Jorge, "LiDAR based detect and avoid system for UAV navigation in UAM corridors," *Drones*, vol. 6, no. 8, p. 185, Aug. 2022. [Online]. Available: https://doi.org/10.3390/drones6080185

[6] Q. Liang, Z. Wang, Y. Yin, W. Xiong, J. Zhang, and Z. Yang, "Autonomous aerial obstacle avoidance using LiDAR sensor fusion," *PLOS ONE*, vol. 18, no. 6, pp. 1–22, Jun. 2023. [Online]. Available: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0287177

[7] Atlassian, "Jira Software." [Online]. Available: https://www.atlassian.com/software/jira

[8] GitHub, Inc., "GitHub: Where the world builds software." [Online]. Available: https://github.com/

[9] Unity Technologies, "Unity Real-Time Development Platform." [Online]. Available: https://unity.com/

[10] Blender Foundation, "Blender: Free and open source 3D creation suite." [Online]. Available: https://www.blender.org/

[11] Google, "Google Drive," Google, Mountain View, CA, USA. [Online]. Available: https://www.google.com/drive/

[12] Anaconda Software Distribution. Anaconda Inc. [Online]. Available: https://www.anaconda.com/

[13] Project Jupyter, "Jupyter Notebook." [Online]. Available: https://jupyter.org/

[14] I. Catalano, X. Yu, and J. P. Queralta, "Towards robust UAV tracking in GNSS-denied environments: a multi-LiDAR multi-UAV dataset," in *Proc. 2023 IEEE Int. Conf. Robot. Biomimetics (ROBIO)*, 2023, pp. 1–7.